

# Computer Architecture Project 1

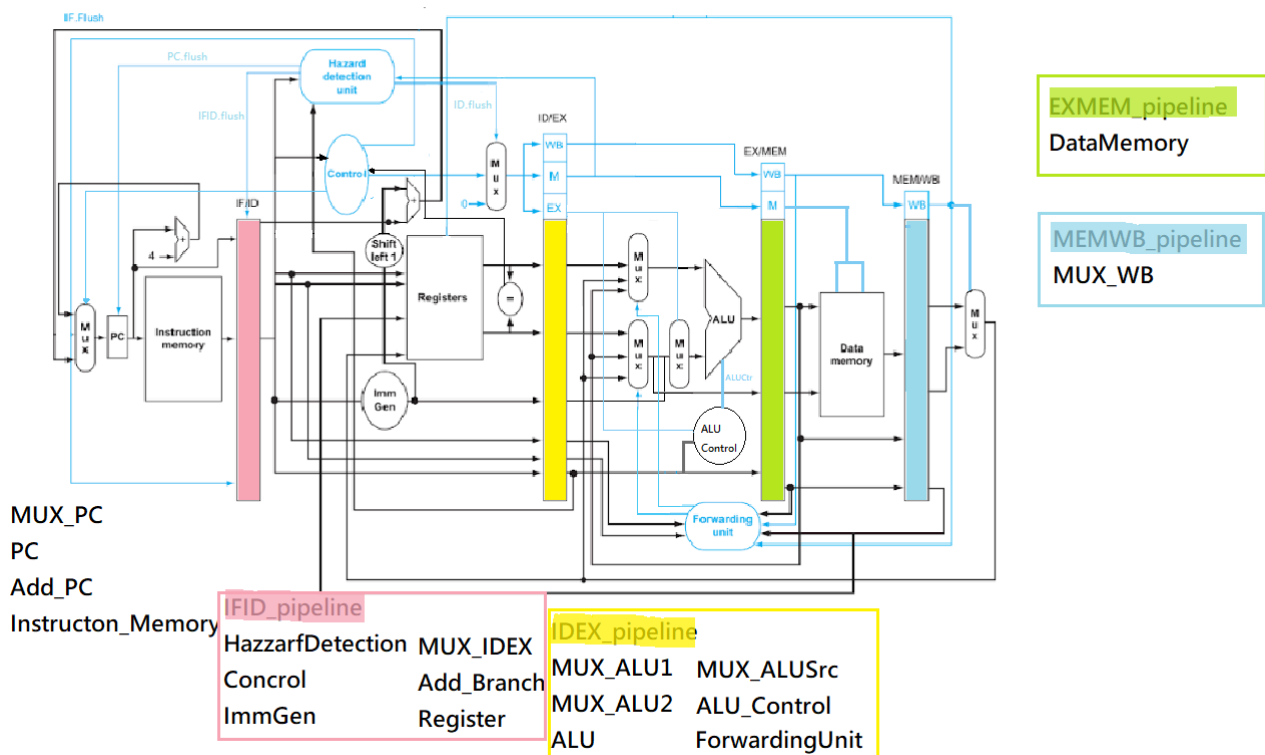
## member and Teamwork

B05902003 李哲安: EX/MEM/WB stages, Forwarding

B05902049 簡崇安: IF/ID stages, Hazard Detection

B05902109 柯上優: testbench, wire connection, report

## Implement pipeline CPU



- 這才是正確且完整的datapath，直到report完成的時間點(2018/12/5 23:00)，作業引導裡付上的圖片都有漏接線路與缺少元件，如：
  - WB/M/EX各自的尾端都沒接完，只停在pipeline裡面。
  - 少了ALU Control Unit和ALU的input data 2前需要一個MUX來保存值。
- 使用clk刺激PC與四個pipeline，posedge會讀入前一個stage傳來的資料，negedge會開始將資料流給右邊的stage。此方法讓五個部分有條理的運行，保證不讓同一個stage同時處理兩個以上的instructions。

## Implement each module

- 比較值得一提，且又是上一次作業沒有處理到的module：
  - Forwarding Unit + 兩個ALU前面的MUX：

- Forwarding功能使的instruction不必stall等待前面的資料寫回Registers。藉由判斷MEM/WB stages要寫回Registers且MEM\_rd/WB\_rd等於EXE\_rs1/2，可以提早將尚未寫回去的資料先拿過來使用。
- Hazard Detection + PC前面的MUX：
  - 此次作業要求ld後面若立刻使用剛load進來的register，則需要stall到資料拿到且可以使用Forwarding提取為止。
  - 判斷的方法是對於要Memory read的instruction，假如接續的instruction就要使用到(IDEX\_rd == IFID\_rs1 || IDEX\_rd == IFID\_rs2)，就傳送信號給「PC」、「IF/ID\_pipeline」、「Control剛算到的E/M/WB」，前者重新跑一次這個instruction，後兩者洗掉，實現stall。
- 增加Jump功能的Control：
  - 此版本的結構，我們在ID stage就先判斷register[rs1] == register[rs2]。若成立，則搶先將Sign Extend的imm左移1、加上此刻的instruction address，傳回PC進行jump。此外，會flush掉提前進入IF stage的一個instruction，取消掉因為我們平行處理而多處理到的部分。

## Problems and Solution

---

- 最一開始沒有pipeline要用posedge和negedge控制寫入寫出的概念，同一個cycle時多個instruction一次執行，造成forwarding與hazard detection嚴重堆積。
  - 由謝議霆與李澤諺同學們的指導下解決問題。
- 同學間的檔案命名品味歧異，造成接線上的疲勞。
  - 接線者最後按照個人命名品味修改其他人寫的port的名稱。
- 李哲安同學在ALU Control的規則表裡面寫錯了addi的ALUSrc型態，造成接線者多de了半小時的bug。
  - 修正並告知他。
- Jump後就停止在跟新instruction address。
  - 發現是Control在jump後不停回傳flush的訊息，修改了一下if-else if-else的邏輯概念後完成所有架構。